

Kubernetes

Container orchestration architecture, workload objects, and security hardening

Overview

Kubernetes automates deployment, scaling, and operations of containerized applications across a cluster of nodes. Understanding its control-plane/data-plane split and core API objects is foundational to securing managed offerings like AKS, EKS, and GKE.

Control Plane Components

	Role
API Server	Front door for all cluster operations; authenticates and validates requests
etcd	Distributed key-value store holding all cluster state — highest-value target to protect
Scheduler	Assigns pods to nodes based on resource requests, affinity, taints/tolerations
Controller Manager	Runs reconciliation loops (ReplicaSet, Node, Endpoint controllers) toward desired state
kubelet	Node agent; ensures containers described in PodSpecs are running and healthy
kube-proxy	Maintains network rules enabling Service abstraction and load balancing

Core Workload Objects

- **Pod** — smallest deployable unit; one or more co-located containers sharing network/storage
- **Deployment** — manages ReplicaSets for stateless apps; supports rolling updates and rollback
- **StatefulSet** — stable network identity and persistent storage per replica, for stateful workloads

- **DaemonSet** — ensures one pod per node (log shippers, node-level security agents)
- **Job / CronJob** — run-to-completion and scheduled batch workloads
- **Service** — stable virtual IP/DNS abstracting a dynamic set of pod endpoints (ClusterIP/NodePort/LoadBalancer)
- **Ingress** — L7 HTTP(S) routing into the cluster, typically backed by an Ingress Controller

Security Hardening Checklist

- **RBAC** — least-privilege Roles/ClusterRoles bound via RoleBindings; avoid cluster-admin for workloads
- **Pod Security Standards** — enforce baseline/restricted profiles (replaces deprecated PodSecurityPolicy)
- **Network Policies** — default-deny ingress/egress, explicitly allow required pod-to-pod traffic
- **Secrets management** — avoid plain Secrets objects for high-value data; integrate external secret stores (Key Vault CSI driver)
- **Image security** — scan images in CI, enforce signed/trusted registries via admission control
- **Admission Controllers** — OPA Gatekeeper / Kyverno to enforce policy-as-code at admission time
- **Workload identity** — bind pod service accounts to cloud IAM roles instead of embedding credentials

Managed Kubernetes Comparison

	Notable Traits
AKS (Azure)	Free control plane; integrates with Entra ID RBAC, Azure Policy, Defender for Containers
EKS (AWS)	IAM Roles for Service Accounts (IRSA), tight VPC CNI integration
GKE (GCP)	Autopilot mode abstracts node management; strong default security posture

kubectl Essentials

	Purpose
<code>kubectl get pods -n ns</code>	List pods in a namespace

<code>kubectl describe pod x</code>	Inspect events, status, resource config
<code>kubectl logs -f pod</code>	Stream container logs
<code>kubectl exec -it pod -- sh</code>	Interactive shell into a container
<code>kubectl apply -f file.yaml</code>	Declarative create/update from manifest
<code>kubectl rollout undo deploy/x</code>	Roll back a deployment

Common Pitfalls

	Fix
Running workloads as root with no security context	Set runAsNonRoot, drop capabilities, enforce restricted Pod Security Standard
Secrets stored unencrypted in etcd	Enable encryption at rest for etcd; consider external secret stores
No resource requests/limits set	Define requests/limits on every container to avoid noisy-neighbor and eviction issues
Flat cluster network, no NetworkPolicy	Default-deny and explicitly allow required pod-to-pod communication
Cluster-admin bound to CI/CD service account	Scope CI/CD RBAC tightly to the namespaces and verbs actually required

Interview / Exam Quick Hits

- etcd should be **encrypted at rest** and access restricted — it holds Secrets in plaintext by default
- Liveness probes restart unhealthy containers; readiness probes control traffic admission, not restarts
- Namespaces provide **logical isolation**, not a hard security boundary — combine with NetworkPolicy + RBAC
- HPA scales pod replicas on metrics; VPA adjusts resource requests/limits; Cluster Autoscaler adds/removes nodes

- Taints repel pods from a node; tolerations allow specific pods to be scheduled there anyway